FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Une application de cette taille meurt par mille petites incohérences : « article-images » écrit avec un tiret ici, un underscore là ; un timeout de 5 secondes recopié à 12 endroits. La centralisation des constantes est l'antidote systématique.*

PHILOSOPHIE

Pourquoi centraliser ?

- **Refactor sans risque** — renommer une table demande de toucher 1 ligne, pas 30.
- **Découverte facilitée** — un nouveau dev voit toutes les routes en un seul fichier.
- **Type-safety** — TypeScript vérifie à la compilation que la clé existe.
- **Documentation implicite** — chaque constante est commentée à un seul endroit.
- **Cohérence i18n** — un seul fichier de timings garantit les mêmes durées partout.

CARTOGRAPHIE

Modules de constantes

MODULE	FICHIER	CONTENU
`ROUTES`	constants/routes.ts	Toutes les URLs (/admin, /savoir/[slug], etc.)
`DB_TABLES`	constants/db.ts	Noms des tables Supabase
`DB_BUCKETS`	constants/db.ts	Noms des buckets Storage
`STORAGE_KEYS`	constants/storage.ts	Clés localStorage (avec préfixe sakata-)
`SESSION_KEYS`	constants/storage.ts	Clés sessionStorage (rares)
`TIMINGS`	constants/timings.ts	Durées (debounce, animations, expirations)
`USER_ROLES`	constants/business.ts	admin, manager, contributor, user
`SUBSCRIPTION_TIERS`	constants/business.ts	free, premium
`APP_VERSION`	constants/business.ts	Version pour purge localStorage

STANDARDS

Règles de discipline

CES RÈGLES SONT OBLIGATOIRES

- Aucune chaîne de caractères représentant une route, table, bucket ou clé localStorage ne doit apparaître en dur dans un composant ou hook.
- Toute nouvelle constante DOIT être ajoutée à son module centralisé.
- Le préfixe `sakata-` est obligatoire pour toute clé localStorage (whitelist du AuthProvider).
- Les helpers de tests de rôle (`canManageContent`, etc.) doivent être utilisés au lieu de comparer manuellement avec `"admin"`.

PATTERNS

Imports recommandés

IMPORTS TYPES VIA BARREL

TYPESCRIPT

```typescript
// ✅ Recommandé
import { ROUTES } from "@/lib/constants/routes";
import { DB_TABLES, DB_BUCKETS } from "@/lib/constants/db";
import { STORAGE_KEYS, msgViewedKey } from "@/lib/constants/storage";
import { TIMINGS } from "@/lib/constants/timings";
import { USER_ROLES, canManageContent } from "@/lib/constants/business";

// ✅ Aussi acceptable (barrel)
import { ROUTES, DB_TABLES, TIMINGS } from "@/lib/constants";
```

VALEUR AJOUTÉE

Cas concrets : avant / après

Renommage du bucket Storage (cas réel)

Lors de la fonctionnalité « Vidéo Hero », il fallait ajouter un bucket `article-videos`. Sans constante, il aurait fallu chercher chaque occurrence du nom dans le code. Avec :

CODE TYPESCRIPT

```typescript
// 1. Une seule modification :
// constants/db.ts
export const DB_BUCKETS = {
  CHAT_ATTACHMENTS: "chat_attachments",
  AVATARS: "avatars",
  ARTICLE_VIDEOS: "article-videos",  // ⟵ AJOUT
};

// 2. Usage type-safe partout :
.from(DB_BUCKETS.ARTICLE_VIDEOS)
```

Bug évité : durée d'expiration de message éphémère

La durée de countdown `VIEW_ONCE_COUNTDOWN` existait à 3 endroits : `ChatInput`, `MessageBubble`, et `ProtectedImage`. Résultat : un changement à un endroit créait des incohérences (image disparaît à 5s mais countdown affiche 8s).

SOLUTION

Centralisé dans `TIMINGS.VIEW_ONCE_COUNTDOWN = 5000`, importé depuis tous les composants concernés.

POUR ALLER PLUS LOIN

Patterns avancés

Helpers de calcul de clé

Pour les clés dynamiques (incluant un ID), on définit des fonctions helpers :

CODE TYPESCRIPT

```
// constants/storage.ts
export const msgViewedKey = (id: string) => `sakata-msg-viewed-${id}`;
export const ecoleProgressKey = (ns: string) => `sakata-ecole-progress-${ns}`;
```

`as const` pour des unions de littéraux

Pour obtenir des types stricts (et l'autocomplétion), utiliser `as const` :

CODE TYPESCRIPT

```
export const USER_ROLES = ["admin", "manager", "contributor", "user"] as const;
export type UserRole = typeof USER_ROLES[number];
// → "admin" | "manager" | "contributor" | "user"
```

À SAVOIR

Limitations connues

- Les noms de colonnes Postgres ne sont PAS dans `DB_TABLES` — tradeoff entre verbosité et utilité.
- Les valeurs CSS / couleurs sont gérées par Tailwind config, pas par ce système.
- L'i18n des chaînes user-facing est dans `LanguageProvider`, pas dans les constantes.

Sakata.com — Documentation interne · feature-constants-architecture · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM